23 Lecture 23: Factorization

23.1 Two theorems in number theory

Theorem 1: if N is a composite (i.e. not prime) L-bit integer, and y is a solution of $y^2 \equiv 1 \pmod{N}$ in the interval $1 < y < N-1$ (thus the trivial solutions $y = 1, y = N-1 \equiv -1 \pmod{N}$ are excluded), then $\mathrm{GCD}(y-1, N)$ and $\mathrm{GCD}(y+1, N)$ are nontrivial factors of N that can be computed with $O(L^3)$ operations.

Proof: since $y^2 \equiv 1 \pmod{N}$, N must divide $y^2 - 1 = (y+1)(y-1)$. Moreover $N > y+1 > y-1$ and therefore N cannot divide $(y+1)$ or $(y-1)$, but must have nontrivial common factors with $(y+1)$ and $(y-1)$. These common factors can be found by computing the GCD of N and $y \pm 1$ with Euclid's algorithm, which requires $O(L^3)$ operations.

Theorem 2: if we choose at random an integer x in the interval $[1, N-1]$, coprime with N, it is very probable that its order $\pmod{N}$ is even, and $x^{r/2}$ is *not* equal to $\pm 1 \pmod{N}$ (the probability is always $\geq 1/2$, see Appendix 4 of Nielsen and Chuang, where this probability is shown to be $1 - \frac{1}{2^m}$ where m is the number of different prime factors of N).

Then $y = x^{r/2}$ (mod N) is a nontrivial solution of $y^2 \equiv 1$ (mod N).

In conclusion, finding the order r modulo N of an integer x, chosen at random in the interval $[1, N-1]$, and coprime with N, allows to find in polynomial time a nontrivial solution of $y^2 \equiv 1 \pmod{N}$, and hence a nontrivial factor of N.

23.2 The factorization algorithm

We have seen that the two theorems of the previous section can be combined to construct a factorization algorithm. This algorithm was first proposed by P.W. Shor in 1994, and produces in polynomial time and with good probability a nontrivial factor of a composite integer N. All operations can be executed on a classical computer, except the order finding subroutine that requires a quantum computer (to find the order in polynomial time).

23.3 Summary of the algorithm

- **Input:** N
- **Output:** a nontrivial factor of N
- **Runtime:** $O((\log_2 N)^3)$ operations
- **Procedure:**

1. If N even. $\longrightarrow$ output $= 2$

2. Determine whether $N = a^b$ for $a \geq 1$, $b \geq 2$ (can be done by a classical algorithm in $O(L^3)$ operations). If yes, $\longrightarrow$ output $= a$.

3. Choose x in the interval $[1, N-1]$. Compute the GCD(x, N) , if it is > 1 then $\longrightarrow$ output $=$ GCD(x, N)

4. Quantum subroutine to find the order r of x modulo N.

5. If r is even and $x^{r/2} \neq -1 \pmod N$, compute GCD$(x^{r/2} - 1, N)$ and GCD$(x^{r/2} + 1, N)$, and test whether these are nontrivial factors of N. If yes, $\longrightarrow$ output $=$ these factors.

NB: $x^{r/2}$ cannot be $= +1 \pmod N$ because r is the *smallest* integer such that $x^r \equiv 1 \pmod N$.

23.4 Example: the factorization of $N = 91$

i) The steps 1. and 2. are passed without exiting.
ii) step 3. : choose for example $x = 4$, coprime with 91.
iii) compute order of 4 modulo 91. Result $= 6$ $(4^6 = 4096 = 1 + 45 \times 91)$.
iv) check that $x^{r/2} \neq -1 \pmod N$, $\rightarrow 4^3 = 64 \neq -1 \pmod{91}$, ok.
v) GCD(64+1, 91) = 13, GCD(64-1, 91) = 7

23.5 Bibliography

P. W. Shor (1994), "Algorithms for quantum computation: discrete log and factoring", Proc. of the 35-th Annual Symposium on the Foundations of Computer Science, 124-134, IEEE Computer Society Press.